

PHP

Plan de cours, initiation à PHP
Préparé par Herizo Randrianarivonjy

PLAN

PLAN

1. Introduction à PHP

1.1. Historique

1.2. Installation et configuration

2. Comment fonctionne PHP

3. Variables et types de variables en PHP

4. Expressions, opérateur et structure de contrôles

4.1. Expressions

4.2. Opérateurs:

4.2.1. La priorité des opérateurs:

4.2.2. Les opérateurs arithmétiques

4.2.3. Les opérateur d'incrément et décrémentation

4.2.4. Les opérateurs d'affectation

4.2.5. Les opérateurs de comparaison

4.2.6. Les opérateurs logiques

4.2.7. Les opérateurs de chaîne de caractères

4.2.8. Opérateurs de tableaux

4.2.9. Opérateurs de types

4.3. Structure de contrôles

4.3.1. If / else / elseif, else if

4.3.2. While

4.3.3. Do while

4.3.4. For

4.3.5. foreach

4.3.6. break

4.3.7. continue

4.3.8. switch

4.3.9. match (dispo à partir de PHP 8.0.0)

4.3.10. require

4.3.11. include

4.3.12. require once

[4.3.13. include_once](#)

[5. Les fonctions](#)

[5.1. Les fonctions définies par l'utilisateur](#)

[5.2. Les arguments de fonction](#)

[5.3. Les valeurs de retour](#)

[5.4. Fonctions variables](#)

[5.5. Fonctions internes](#)

[5.6. Fonctions anonymes](#)

[5.7. Fonction fléchée](#)

[6. Tableau en PHP](#)

[7. PHP Orienté objet](#)

[7.1. Les classes](#)

[7.1.1. Les propriétés et méthodes](#)

[7.1.2. Les constantes de classe](#)

[7.1.3. Visibilité](#)

[7.1.4. Héritage](#)

[7.1.5. Statique](#)

[7.1.6. Abstraction de classes](#)

[7.1.7. Interfaces](#)

[7.1.8. Traits](#)

[7.1.9. Parcours d'objets](#)

[8. Formulaire avec PHP](#)

[9. Base de données MySQL](#)

[10. Interaction avec une base de données MySQL avec PHP](#)

[10.1. Connecter PHP sur MySQL](#)

1. Introduction à PHP

1.1. Historique

Le langage PHP fut créé en 1994 par Rasmus Lerdorf pour son site web. A ses débuts c'était une bibliothèque logicielle écrite en langage Perl et puis en langage C.

Depuis il y a plusieurs versions du langage qui se sont succédées, avec des corrections et améliorations. La version 5 est sortie en 2004, la dernière version stable PHP 8.2.8 est sortie le 26 Juillet 2023.

1.2. Installation et configuration

Download laragon <https://laragon.org/download/>

2. Comment fonctionne PHP

PHP est un langage script, ce qui signifie que le code est interprété et non compilé comme le langage C ou C++. Cela permet de détacher le code php du type d'OS sur lequel il doit être interprété.

Un langage compilé: la machine cible traduit directement le programme.

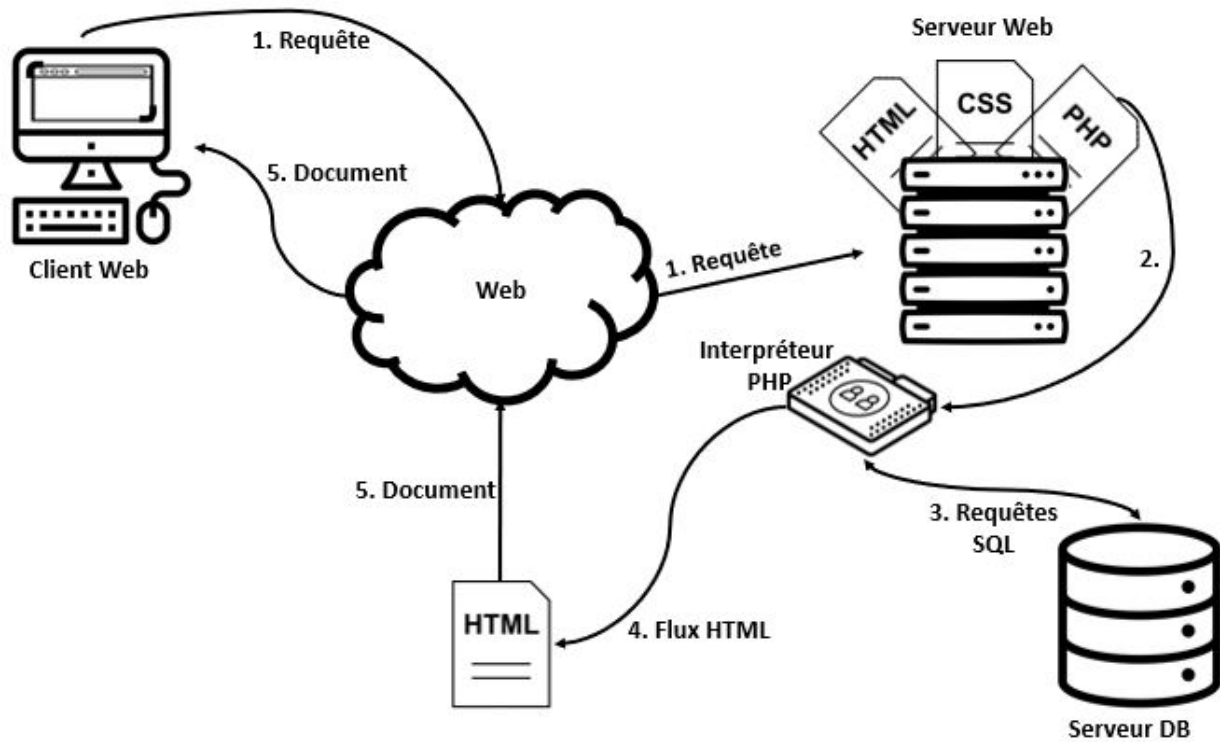
Un langage interprété: Le code n'est pas directement traduit par la machine cible mais un programme différent appelé l'interpréteur lit et exécute le code.

Il est possible d'utiliser PHP en mode ligne de commande; cependant PHP est principalement associé à un serveur Web utilisant le protocole HTTP tout cela dans une architecture client/serveur.

Un serveur web peut être composé d'un système d'exploitation, un serveur HTTP, un langage serveur et enfin un système de gestion de base de données.

Dans le cas de PHP voici les combinaisons les plus courantes:

1. Plateforme LAMP pour Linux Apache Mysql PHP;
2. WAMP: WIndows Apache MySQL PHP
3. MAMP: Mac Os Apache MySQL PHP



3. Variables et types de variables en PHP

Voici la liste des types de base:

- Types intégrés:
 - type null;
 - Types scalaires:
 - type bool;
 - type int;
 - type float;
 - type string;
 - type array;
 - type object;
 - type resource;
 - type never;
 - type void;
 - type de classes relatives: self, parent, static;
- Type littéraux:
 - false;
 - true;

- Type définis par l'utilisateur:
 - Interfaces;
 - Classes;
 - Enumérations
- Type callable;

4. Expressions, opérateur et structure de contrôles

4.1. Expressions

En PHP presque tout ce qu'on écrit est une expression. Par définition c'est tout ce qui a une valeur.

Les constantes et les variables sont des exemples d'expression. Par exemple:

`$a = 5;` Ici vous assignez la valeur 5 à la variable `$a`;

`$b = 5;` Idem

Après cette assignation la valeur de `$a` est 5 et donc si on fait l'affectation `$b = $a` ce qui revient à écrire `$b = 5;`

Les fonctions sont des exemples plus complexes d'expression en PHP.

```
<?php
function foo ()
{
    return 5;
}
?>
```

La *pré-incrémentation* et la *post-incrémentation* sont d'autres exemples d'expression.

PRE-INCREMENTATION:

`++$variable` PHP incrémente la variable avant de lire la valeur de cette variable, d'où le nom pré-incrémentation.

POST-INCREMENTATION:

`$variable++` PHP incrémente la variable après avoir lu sa valeur, d'où le nom post-incrémentation.

Opérateur ternaire:

C'est une expression qui peut paraître complexe mais très utile pour raccourcir votre code.

```
$a = $b === 1? $b : 0;
```

4.2. Opérateurs:

4.2.1. La priorité des opérateurs:

La priorité des opérateurs spécifie l'ordre dans lequel les valeurs doivent être analysées.

Par exemple:

`1 + 5 * 3` : le résultat est 16 et non 18 car la multiplication a une priorité supérieure par rapport à l'addition. Des parenthèses peuvent être utilisées pour forcer la priorité, si nécessaire. Par exemple: `(1 + 5) * 3` donnera 18.

En PHP:

```
$a = 1 + 5 * 3;
```

```
echo $a; //affichera 16
```

4.2.2. Les opérateurs arithmétiques

Exemple	Nom	Résultat
+\$a	Identité	Conversion de \$a vers int ou float, selon le plus approprié.
-\$a	Négation	Opposé de \$a.
\$a + \$b	Addition	Somme de \$a et \$b.
\$a - \$b	Soustraction	Différence de \$a et \$b.
\$a * \$b	Multiplication	Produit de \$a et \$b.
\$a / \$b	Division	Quotient de \$a et \$b.
\$a % \$b	Modulus	Reste de \$a divisé par \$b.
\$a ** \$b	Exponentiation	Résultat de l'élévation de \$a à la puissance \$b.

4.2.3. Les opérateur d'incrément et décrémentation

PHP prend en charge les opérateurs de pre et post incrément et décrémentation.

Exemple	Nom	Résultat
++\$a	Pre-incrémente	Incrémente \$a de 1, puis retourne \$a.
\$a++	Post-incrémente	Retourne \$a, puis incrémente \$a de 1.
--\$a	Pré-décrémente	Décrémente \$a de 1, puis retourne \$a.
\$a--	Post-décrémente	Retourne \$a, puis décrémente \$a de 1.

4.2.4. Les opérateurs d'affectation

L'opérateur d'affectation est le signe "=", cependant ce signe ne veut pas dire "égal à" mais signifie que l'opérande de gauche se voit affecter la valeur de l'expression à droite du signe égal.

```
<?php
$a = ($b = 4) + 5;
// $a est maintenant égal à 9, et $b vaut 4.
?>
```

Il existe aussi des opérateurs combinés pour tous les opérateurs arithmétiques.

```
<?php
$a = 3;
$a += 5; // affecte la valeur 8 à la variable $a correspond à
l'instruction '$a = $a + 5';
$b = "Bonjour ";
$b .= " tout le monde!"; // affecte la valeur "Bonjour tout le
monde!" à
                        // la variable $b
                        // identique à $b = $b." tout le monde!";

?>
```

On peut constater que l'affectation copie la valeur d'une variable originale dans la nouvelle variable. Par conséquent, la modification de la valeur de la variable originale n'affectera pas la valeur de la nouvelle.

Affectation par référence:

L'affectation par référence se fait par l'utilisation du caractère &:

```
<?php
$a = 3;
$b = &$a; // $b est une référence à $a

print "$a\n"; // affiche 3
print "$b\n"; // affiche 3

$a = 4; // change $a

print "$a\n"; // affiche 4
print "$b\n"; // affiche 4 aussi, car $b est une référence à $a, qui
a été
                // changée

?>
```

Les opérateurs d'affectation arithmétiques

Exemple	Equivalent	Opération
\$a += \$b	\$a = \$a + \$b	Addition
\$a -= \$b	\$a = \$a - \$b	Soustraction
\$a *= \$b	\$a = \$a * \$b	Multiplication
\$a /= \$b	\$a = \$a / \$b	Division
\$a %= \$b	\$a = \$a % \$b	Modulo
\$a **= \$b	\$a = \$a ** \$b	Exponentiation

Les opérateurs d'affectation bits à bits

Exemple	Equivalent	Opération
<code>\$a &= \$b</code>	<code>\$a = \$a & \$b</code>	Opérateur And
<code>\$a = \$b</code>	<code>\$a = \$a \$b</code>	Opérateur Or
<code>\$a ^= \$b</code>	<code>\$a = \$a ^ \$b</code>	Opérateur Xor
<code>\$a <<= \$b</code>	<code>\$a = \$a << \$b</code>	Décalage à gauche
<code>\$a >>= \$b</code>	<code>\$a = \$a >> \$b</code>	Décalage à droite

Autres opérateurs d'affectation

Exemple	Equivalent	Opération
<code>\$a .= \$b</code>	<code>\$a = \$a . \$b</code>	Concaténation d'une chaîne de caractères
<code>\$a ??= \$b</code>	<code>\$a = \$a ?? \$b</code>	Opérateur de coalescence nul

4.2.5. Les opérateurs de comparaison

Les opérateurs de comparaison permettent de comparer deux valeurs.

Exemple	Nom	Résultat
<code>\$a == \$b</code>	Égal	true si <code>\$a</code> est égal à <code>\$b</code> après le transtypage.
<code>\$a === \$b</code>	Identique	true si <code>\$a</code> est égal à <code>\$b</code> et qu'ils sont de même type.
<code>\$a != \$b</code>	Différent	true si <code>\$a</code> est différent de <code>\$b</code> après le transtypage.
<code>\$a <> \$b</code>	Différent	true si <code>\$a</code> est différent de <code>\$b</code> après le transtypage.
<code>\$a !== \$b</code>	Différent	true si <code>\$a</code> est différent de <code>\$b</code> ou bien s'ils ne sont pas du même type.
<code>\$a < \$b</code>	Plus petit que	true si <code>\$a</code> est strictement plus petit que <code>\$b</code> .
<code>\$a > \$b</code>	Plus grand	true si <code>\$a</code> est strictement plus grand que <code>\$b</code> .
<code>\$a <= \$b</code>	Inférieur ou égal	true si <code>\$a</code> est plus petit ou égal à <code>\$b</code> .
<code>\$a >= \$b</code>	Supérieur ou égal	true si <code>\$a</code> est plus grand ou égal à <code>\$b</code> .
<code>\$a <=> \$b</code>	Combiné	Un <u>entier</u> inférieur, égal ou supérieur à zéro lorsque <code>\$a</code> est inférieur, égal, ou supérieur à <code>\$b</code> respectivement.

Si on compare des opérandes qui sont des chaînes numériques ou si une opérande est un nombre et l'autre est une chaîne numérique, alors la comparaison sera effectuée numériquement.

4.2.6. Les opérateurs logiques

Exemple	Nom	Résultat
<code>\$a and \$b</code>	And (Et)	true si <code>\$a</code> ET <code>\$b</code> valent true .
<code>\$a or \$b</code>	Or (Ou)	true si <code>\$a</code> OU <code>\$b</code> valent true .
<code>\$a xor \$b</code>	XOR	true si <code>\$a</code> OU <code>\$b</code> est true , mais pas les deux en même temps.
<code>! \$a</code>	Not (Non)	true si <code>\$a</code> n'est pas true .
<code>\$a && \$b</code>	And (Et)	true si <code>\$a</code> ET <code>\$b</code> sont true .
<code>\$a \$b</code>	Or (Ou)	true si <code>\$a</code> OU <code>\$b</code> est true .

Il y a deux types de ET et de OU car ils ont des priorités différentes. `&&` possède une priorité plus élevée que "and"

4.2.7. Les opérateurs de chaîne de caractères

Il existe deux opérateurs de chaînes de caractères "." qui est l'opérateur de concaténation et le second c'est l'opérateur d'affectation concaténant (.=)

```
<?php
$a = "Bonjour ";
$b = $a . "Monde !"; // $b contient "Bonjour Monde !"

$a = "Bonjour ";
$a .= "Monde !"; // $a contient "Bonjour Monde !"
?>
```

4.2.8. Opérateurs de tableaux

Exemple	Nom	Résultat
<code>\$a + \$b</code>	Union	Union de <code>\$a</code> et <code>\$b</code> .
<code>\$a == \$b</code>	Égalité	true si <code>\$a</code> et <code>\$b</code> contiennent les mêmes paires clés/valeurs.
<code>\$a === \$b</code>	Identique	true si <code>\$a</code> et <code>\$b</code> contiennent les mêmes paires clés/valeurs dans le même ordre et du même type.
<code>\$a != \$b</code>	Inégalité	true si <code>\$a</code> n'est pas égal à <code>\$b</code> .
<code>\$a <> \$b</code>	Inégalité	true si <code>\$a</code> n'est pas égal à <code>\$b</code> .
<code>\$a !== \$b</code>	Non-identique	true si <code>\$a</code> n'est pas identique à <code>\$b</code> .

Voici quelques exemples

```
<?php
$a = array("a" => "pomme", "b" => "banane");
$b = array("a" => "poire", "b" => "fraise", "c" => "cerise");

$c = $a + $b; // Union de $a et $b
echo "Union de \$a et \$b : \n";
var_dump($c);

$c = $b + $a; // Union de $b et $a
echo "Union de \$b et \$a : \n";
var_dump($c);

$a += $b; // Union de $a += $b est $a and $b
echo "Union de \$a += \$b: \n";
var_dump($a);
?>
```

A l'exécution, le script affichera:

```
Union de $a et $b :
array(3) {
    ["a"]=>
    string(5) "pomme"
    ["b"]=>
    string(6) "banane"
    ["c"]=>
    string(6) "cerise"
}
Union de $b et $a :
array(3) {
    ["a"]=>
    string(5) "poire"
    ["b"]=>
    string(6) "fraise"
    ["c"]=>
    string(6) "cerise"
}
Union de $a += $b:
array(3) {
    ["a"]=>
    string(5) "pomme"
    ["b"]=>
    string(6) "banane"
    ["c"]=>
    string(6) "cerise"
}
```

4.2.9. Opérateurs de types

Le mot clé utilisé c'est `instanceof` pour déterminer si une variable PHP est un objet instancié d'une certaine classe.

```
<?php
class MaClasse
{
}
class PasMaClasse
{
}
$a = new MaClasse;

var_dump($a instanceof MaClasse);
var_dump($a instanceof PasMaClasse);
?>
```

L'exemple ci-dessus va afficher :

```
bool(true)
bool(false)
```

4.3. Structure de contrôles

4.3.1. If / else / elseif, else if

L'instruction `if` permet l'exécution conditionnelle d'une partie du code.

```
<?php
if ($a > $b)
    echo "a est plus grand que b";
?>
```

L'instruction "else" est utilisée si on veut exécuter une instruction si une condition est remplie et une autre instruction si cette condition n'est pas remplie.

```
<?php
if ($a > $b) {
    echo "a est plus grand que b";
} else {
    echo "a est plus petit que b";
}
?>
```

Les instructions dans le else ne sont exécutées que si l'expression dans le if n'est pas vérifiée.

L'instruction "elseif" permet d'exécuter une instruction après un if dans le cas où le premier if n'est pas vérifié. Mais la différence avec l'expression else c'est que l'instruction ne s'exécutera que si la condition dans le elseif est vérifiée.

```
<?php
if ($a > $b) {
    echo "a est plus grand que b";
} elseif ($a == $b) {
    echo "a est égal à b";
} else {
    echo "a est plus petit que b";
}
?>
```

Note: L'utilisation de elseif et else if est la même quand les accolades sont utilisées mais quand vous utilisez ":" PHP échouera avec une erreur d'analyse.

4.3.2. While

Le boucle while est le moyen le plus simple d'implémenter une boucle en PHP:

while (expression)

commandes

PHP exécutera les scripts (commandes) tant que la boucle while est évalué comme true.

```
<?php
/* exemple 1 */

$i = 1;
while ($i <= 10) {
    echo $i++; /* La valeur affichée est $i avant l'incrémentation
                (post-incrémentation) */
}

/* exemple 2 */

$i = 1;
while ($i <= 10):
    echo $i;
    $i++;
endwhile;
?>
```

4.3.3. Do while

Les boucles **do-while** ressemblent beaucoup aux boucles **while**, mais l'expression est testée à la fin de chaque itération plutôt qu'au début. La première itération de la boucle **do-while** est toujours exécutée.

```
<?php
$i = 0;
do {
    echo $i;
} while ($i > 0);
?>
```

4.3.4. For

Les boucles **for** sont les boucles les plus complexes en PHP.

for (expr1; expr2; expr3)

commandes

La première expression (**expr1**) est évaluée (exécutée), quoi qu'il arrive au début de la boucle.

Au début de chaque itération, l'expression **expr2** est évaluée, si **true** la boucle continue sinon l'exécution de la boucle s'arrête.

A la fin de chaque itération, l'expression **expr3** est évaluée (exécutée).

```
for ($i = 1; $i <= 10; $i++) {
    echo $i;
}
```

4.3.5. foreach

La structure de langage `foreach` fournit une façon simple de parcourir des tableaux. `foreach` ne fonctionne que pour les tableaux et les objets.

```
foreach ( iterable_expression as $value) {  
  
    //commandes  
  
}
```

```
foreach ( iterable_expression as $key => $value) {  
  
    //commandes  
  
}
```

```
<?php  
$arr = array(1, 2, 3, 4);  
foreach ($arr as &$value) {  
    $value = $value * 2;  
}  
// $arr vaut maintenant array(2, 4, 6, 8)  
unset($value); // Détruit la référence sur le dernier élément  
?>
```

4.3.6. break

L'instruction `break` permet de sortir d'une structure `for`, `while`, `do-while` ou `switch`.

```
<?php
$arr = array('un', 'deux', 'trois', 'quatre', 'stop', 'cinq');
foreach ($arr as $val) {
    if ($val == 'stop') {
        break;    /* Vous pourriez aussi utiliser 'break 1;' ici. */
    }
    echo "$val<br />\n";
}

/* Utilisation de l'argument optionnel. */

$i = 0;
while (++$i) {
    switch ($i) {
        case 5:
            echo "At 5<br />\n";
            break 1; /* Termine uniquement le switch. */
        case 10:
            echo "At 10; quitting<br />\n";
            break 2; /* Termine le switch et la boucle while. */
        default:
            break;
    }
}
?>
```

4.3.7. continue

L'instruction continue permet de passer à la prochaine itération et d'ignorer les instructions après continue.

```
<?php
$arr = ['zero', 'one', 'two', 'three', 'four', 'five', 'six'];
foreach ($arr as $key => $value) {
    if (0 === ($key % 2)) { // évite les membres pairs
        continue;
    }
    echo $value . "\n";
}
?>
```

Les exemples ci-dessus vont afficher :

```
one
three
five
```

4.3.8. switch

L'instruction `switch` équivaut à une série d'instructions `if`. Par conséquent, on l'utilise quand on a besoin de comparer la même variable avec un grand nombre de valeurs différentes.

```
<?php
// Ce switch:

switch ($i) {
    case 0:
        echo "i égal 0";
        break;
    case 1:
        echo "i égal 1";
        break;
    case 2:
        echo "i égal 2";
        break;
}

// Équivaut à:

if ($i == 0) {
    echo "i égal 0";
} elseif ($i == 1) {
    echo "i égal 1";
} elseif ($i == 2) {
    echo "i égal 2";
}
?>
```

PHP exécutera chacune des clauses dans l'ordre. Si une instruction `case` est vérifiée PHP exécutera les instructions correspondantes. PHP continue d'exécuter les instructions jusqu'à la fin du bloc ou bien dès qu'il trouve l'instruction `break`.

Dans l'exemple suivant, si `$i` est égal à 0, PHP va exécuter toutes les instructions qui suivent! Si `$i` est égal à 1, PHP exécutera les deux dernières instructions.

```
<?php
switch ($i) {
    case 0:
        echo "i égal 0";
    case 1:
        echo "i égal 1";
    case 2:
        echo "i égal 2";
}
?>
```

On peut aussi combiner les cases de la manière suivantes:

```
<?php
switch ($i) {
    case 0:
    case 1:
    case 2:
        echo "i est plus petit que 3 mais n'est pas négatif";
        break;
    case 3:
        echo "i égal 3";
}
?>
```

Si aucun des case est valide il est possible d'ajouter un cas spécial **default**

```
<?php
switch ($i) {
    case 0:
        echo "i égal 0";
        break;
    case 1:
        echo "i égal 1";
        break;
    case 2:
        echo "i égal 2";
        break;
    default:
        echo "i n'est ni égal à 2, ni à 1, ni à 0.";
}
?>
```

4.3.9. match (dispo à partir de PHP 8.0.0)

L'expression **match** permet d'effectuer une évaluation basée sur le contrôle d'identité d'une valeur. Contrairement à **switch**, elle s'évalue à une valeur, comme les expressions ternaires, la comparaison est une vérification d'identité (===) plutôt qu'un contrôle d'égalité faible (==).

```
<?php
$food = 'cake';

$return_value = match ($food) {
    'apple' => 'This food is an apple',
    'bar' => 'This food is a bar',
    'cake' => 'This food is a cake',
};

var_dump($return_value);
?>
```

L'exemple ci-dessus va afficher :

```
string(19) "This food is a cake"
```


4.3.10. require

`require` est identique à `include` sauf que si une erreur survient il produit également une erreur fatale de niveau `E_COMPILE_ERROR`, c'est à dire qu'il stoppera le script alors que `include` émettra une alerte de niveau `E_WARNING`.

```
<?php
require 'somefile.php';
?>
```

4.3.11. include

L'expression de langage `include` inclut et exécute le fichier spécifié en argument.

```
vars.php
<?php

$couleur = 'verte';
$fruit = 'pomme';

?>

test.php
<?php

echo "Une $fruit $couleur"; // Une

include 'vars.php';

echo "Une $fruit $couleur"; // Une pomme verte

?>
```

4.3.12. require_once

L'expression `require_once` est identique à `require` sauf que PHP vérifie si le fichier a déjà été inclus, si c'est le cas on ne l'inclut pas une deuxième fois.

```

<?php

define('__ROOT__', dirname(dirname(__FILE__)));

require_once(__ROOT__.'/config.php');

?>

```

4.3.13. include_once

L'instruction `include_once` inclut et évalue le fichier spécifié durant l'exécution du script. Le comportement est similaire à "`include`", mais la différence est que si le code a déjà été inclus, il ne le sera pas une seconde fois, et `include_once` retourne true. Comme son nom l'indique, le fichier sera inclus une seule fois.

5. Les fonctions

5.1. Les fonctions définies par l'utilisateur

Toutes les fonctions et classes en PHP ont une portée globale. Tout code PHP, correct peut apparaître dans une fonction. Une fonction peut être définie de la manière suivante:

```

<?php
function foo($arg_1, $arg_2, /* ..., */ $arg_n)
{
    echo "Exemple de fonction.\n";
    return $retval;
}

?>

```

La norme d'écriture d'une fonction PHP est la suivante: *Un nom de fonction valide commence par une lettre ou un souligné, suivi par un nombre quelconque de lettres, de nombre ou de soulignés.*

Les fonctions conditionnelles:

Les fonctions peuvent être utilisées avant d'être définies sauf lorsqu'une fonction est définie de manière conditionnelle comme dans l'exemple suivant:

```
<?php

$makefoo = true;

/* Impossible d'appeler foo() ici,
   car cette fonction n'existe pas.
   Mais nous pouvons utiliser bar() */

bar();

if ($makefoo) {
    function foo()
    {
        echo "Je n'existe pas tant que le programme n'est pas passé
ici.\n";
    }
}

/* Maintenant, nous pouvons appeler foo()
   car $makefoo est évalué à vrai */

if ($makefoo) foo();

function bar()
{
    echo "J'existe dès le début du programme.\n";
}

?>
```

Fonction dans une autre fonction:

PHP ne supporte pas la surcharge, la destruction et la redéfinition de fonctions déjà déclarées.

```
<?php
function foo()
{
    function bar()
    {
        echo "Je n'existe pas tant que foo() n'est pas appelé.\n";
    }
}

/* Impossible d'appeler bar() ici
   car il n'existe pas. */

foo();

/* Maintenant, nous pouvons appeler bar(),
   car l'utilisation de foo() l'a rendue
   accessible. */

bar();

?>
```

5.2. Les arguments de fonction

On peut passer des arguments à une fonction séparée par des virgules. Les arguments seront évalués depuis la gauche vers la droite, avant que la fonction soit actuellement appelée.

Il est possible de passer des arguments par valeur (comportement par défaut), par référence et des valeurs par défaut.

Variable d'argument sous forme de tableau.

```
<?php
function takes_array($input)
{
    echo "$input[0] + $input[1] = ", $input[0]+$input[1];
}
?>
```

Passage d'argument par référence :

Par défaut, les arguments sont passés à la fonction par valeur. Par conséquent, changer la valeur d'un argument dans la fonction ne change pas sa valeur à l'extérieur contrairement au passage d'argument par référence.

```
<?php
function add_some_extra(&$string)
{
    $string .= ', et un peu plus.';
}
$str = 'Ceci est une chaîne';
add_some_extra($str);
echo $str; // Affiche : 'Ceci est une chaîne, et un peu plus.'
?>
```

Valeur par défaut des arguments:

Une fonction peut définir des valeurs par défaut pour les arguments.

```
<?php
function servir_cafe ($type = "cappuccino")
{
    return "Servir un $type.\n";
}
echo servir_cafe();
echo servir_cafe(null);
echo servir_cafe("espresso");
?>
```

L'exemple ci-dessus va afficher :

```
Servir un cappuccino.
Servir un .
Servir un espresso.
```

N.B: Les arguments avec des valeurs par défaut doivent obligatoirement être spécifiés après les arguments obligatoires.

La valeur par défaut d'un argument doit être une constante, et ne peut être ni une variable, ni un membre de classe, ni un appel de fonction.

```
<?php
function faireunyaourt ($flavour, $container = "bol")
{
    return "Préparer un $container de yaourt à la $flavour.\n";
}

echo faireunyaourt ("framboise"); // "framboise" est $flavour
?>
```

Liste d'argument à nombre variable

En PHP il est possible d'avoir un nombre variable d'argument dans les fonctions en utilisant "..."

```
<?php
function sum(...$numbers) {
    $acc = 0;
    foreach ($numbers as $n) {
        $acc += $n;
    }
    return $acc;
}

echo sum(1, 2, 3, 4);
?>
```

L'exemple ci-dessus va afficher :

10

Arguments nommés:

Les arguments nommés permettent de passer les arguments à une fonction en s'appuyant sur le nom du paramètre, au lieu de la position du paramètre.

```
<?php
function foo($a, $b, $c = 3, $d = 4) {
    return $a + $b + $c + $d;
}
var_dump(foo(...[1, 2], d: 40)); // 46
var_dump(foo(...['b' => 2, 'a' => 1], d: 40)); // 46
var_dump(foo(...[1, 2], b: 20)); // Erreur fatale. Le paramètre nommé
$b écrase l'argument précédent.
?>
```

5.3. Les valeurs de retour

Ce sont les valeurs renvoyées avec une instruction de retour “return”. Tous les types de variables peuvent être renvoyés, tableaux et objets compris.

```
<?php
function carre($num)
{
    return $num * $num;
}
echo carre(4); // Affiche '16'
?>
```


Une fonction ne peut pas renvoyer plusieurs valeurs en même temps mais on peut obtenir le même résultat en renvoyant un tableau.

```
<?php
function petit_nombre()
{
    return [0, 1, 2];
}
// La déconstruction d'un tableau collectera chaque membre du tableau
individuellement
[$zero, $one, $two] = petit_nombre();

// Antérieur à PHP 7.1, la seule alternative équivalente est en
utilisant la structure de langage list()
list ($zero, $un, $deux) = petit_nombre();
?>
```

5.4. Fonctions variables

Si le nom d'une variable est suivi de parenthèses, PHP recherchera une fonction de même nom et essaiera de l'exécuter cela pourra servir pour faire des fonctions de rappel, des tables de fonctions...

```
<?php
function foo() {
    echo "dans foo()<br />\n";
}

function bar($arg = '')
{
    echo "Dans bar(); l'argument était '$arg'.<br />\n";
}

// Ceci est une fonction détournée de echo
function echoit($string)
{
    echo $string;
}

$func = 'foo';
$func();           // Appel foo()

$func = 'bar';
$func('test');    // Appel bar()

$func = 'echoit';
$func('test');    // Appel echoit()
?>
```

5.5. Fonctions internes

PHP dispose de nombreuses fonctions et structures standards. Il y a des fonctions qui requièrent des extensions spécifiques de PHP. Par exemple pour utiliser les fonctions d'images, telles que *imagecreatetruecolor()* on aura besoin de l'extension GD de PHP.

```
<?php
var_dump(strlen(null));
// "Deprecated: Passing null to parameter #1 ($string) of type
string is deprecated" as of PHP 8.1.0
// int(0)

var_dump(str_contains("foobar", null));
// "Deprecated: Passing null to parameter #2 ($needle) of type
string is deprecated" as of PHP 8.1.0
// bool(true)
?>
```

5.6. Fonctions anonymes

Les fonctions anonymes, aussi appelées *fermetures* ou *closures* permettent la création de fonctions sans préciser leur nom.

Elles sont très utiles comme fonction de rappel callable.

```
<?php
$greet = function($name) {
    printf("Bonjour %s\r\n", $name);
};

$greet('World');
$greet('PHP');
?>
```

5.7. Fonction fléchée

Les fonctions fléchées sont plus concises que les fonctions anonymes. Cependant, les deux supportent les mêmes fonctionnalités, à l'exception que l'utilisation des variables de la portée parent est automatique.

```
<?php

$y = 1;

$fn1 = fn($x) => $x + $y;
// equivalent to using $y by value:
$fn2 = function ($x) use ($y) {
    return $x + $y;
};

var_export($fn1(3));
?>
```

l'exemple ci-dessus affichera 4

6. Tableau en PHP

Un tableau en PHP est un type qui associe des valeurs à des clés. La structure de langage `array()` est utilisée pour créer un tableau. Exemple:

```
array(  
    key => value,  
    key2 => value2,  
    ...  
)
```

Une syntaxe plus courte pour créer un tableau est de remplacer `array()` par `[ ]`.

```
<?php  
$array = array(  
    "foo" => "bar",  
    "bar" => "foo",  
);  
  
// Utilisant la syntaxe de tableau courte  
$array = [  
    "foo" => "bar",  
    "bar" => "foo",  
];  
?>
```

7. PHP Orienté objet

7.1. Les classes

Une classe est une définition qui contient le nom des propriétés ainsi que des méthodes. Les propriétés sont des variables internes dans lesquelles on stocke des valeurs.

Instancier une classe permet d'avoir un objet. Une définition de classe basique commence par le mot clé `class` suivi du nom de la classe.

```
<?php
class SimpleClass
{
    // déclaration d'une propriété
    public $var = 'une valeur par défaut';

    // déclaration des méthodes
    public function displayVar() {
        echo $this->var;
    }
}
?>
```

`$this` est la valeur de l'objet appelant.

Pour créer une instance d'une classe on utilise le mot clé `new`. *Les classes doivent être définies avant l'instanciation.*

```
<?php
class Test
{
    static public function getNew()
    {
        return new static;
    }
}

class Child extends Test
{}

$obj1 = new Test();
$obj2 = new $obj1;
var_dump($obj1 !== $obj2);

$obj3 = Test::getNew();
var_dump($obj3 instanceof Test);

$obj4 = Child::getNew();
var_dump($obj4 instanceof Child);
?>
```

7.1.1. Les propriétés et méthodes

Les variables dans une classe sont appelées **propriétés**. Les propriétés non statiques peuvent être appelées en utilisant la syntaxe `->`. Par contre, les propriétés statiques peuvent être appelées en utilisant la syntaxe `::`:

Exemple:

`self::$property;`

```
<?php
class SimpleClass
{
    public $var1 = 'hello ' . 'world';
    public $var2 = <<<EOD
hello world
EOD;
    public $var3 = 1+2;
    // déclaration de propriété invalide :
    public $var4 = self::myStaticMethod();
    public $var5 = $myVar;

    // Déclarations valides de propriétés :
    public $var6 = myConstant;
    public $var7 = [true, false];
    public $var8 = <<<'EOD'
hello world
EOD;

    // Sans modificateur de visibilité :
    static $var9;
    readonly int $var10;
}
?>
```


Les définitions des propriétés peuvent avoir une déclaration de type.

```
<?php

class User
{
    public int $id;
    public ?string $name;

    public function __construct(int $id, ?string $name)
    {
        $this->id = $id;
        $this->name = $name;
    }
}

$user = new User(1234, null);

var_dump($user->id);
var_dump($user->name);

?>
```

L'exemple ci-dessus va afficher :

```
int(1234)
NULL
```

Les propriétés en lecture seule avec readonly

```
<?php

class Test {
    public readonly string $prop;

    public function __construct(string $prop) {
        // Initialization légale.
        $this->prop = $prop;
    }
}

$test = new Test("foobar");
// Lecture légale.
var_dump($test->prop); // string(6) "foobar"

// Réaffectation illégal. Ce n'importe pas que la valeur
// assigné soit identique.
$test->prop = "foobar";
// Error: Cannot modify readonly property Test::$prop
?>
```

Note: les propriétés statiques en lecture seule ne sont pas supportées.

Les propriétés en lecture seule sont disponibles à partir de PHP 8.1.0. Le modificateur readonly empêche la modification de la propriété après initialisation.

```
<?php

class Test {
    public readonly string $prop;

    public function __construct(string $prop) {
        // Initialization légale.
        $this->prop = $prop;
    }
}

$test = new Test("foobar");
// Lecture légale.
var_dump($test->prop); // string(6) "foobar"

// Réaffectation illégal. Ce n'importe pas que la valeur
// assigné soit identique.
$test->prop = "foobar";
// Error: Cannot modify readonly property Test::$prop
?>
```

7.1.2. Les constantes de classe

Il est possible d'avoir des constantes dans une classe. Il est important de noter que les constantes sont allouées une fois par classe et non pour chaque instance de classe.

```
<?php
class MyClass
{
    const CONSTANT = 'valeur constante';

    function showConstant() {
        echo self::CONSTANT . "\n";
    }
}

echo MyClass::CONSTANT . "\n";

$classname = "MyClass";
echo $classname::CONSTANT . "\n";

$class = new MyClass();
$class->showConstant();

echo $class::CONSTANT . "\n";
?>
```

7.1.3. Visibilité

la visibilité d'une propriété, d'une méthode, une constante est définie avec un mot-clé: **public**, **protected** ou **private**.

Les propriétés déclarées comme **public** sont accessibles partout. Par contre, les propriétés déclarées comme **protected** sont accessibles seulement dans la classe elle-même et dans les classes qui en héritent et parents. Les propriétés déclarées comme **private** sont uniquement réservées à la classe qui les a définis.

```
<?php
/**
 * Définition de MyClass
 */
class MyClass
{
    public $public = 'Public';
    protected $protected = 'Protected';
    private $private = 'Private';

    function printHello()
    {
        echo $this->public;
        echo $this->protected;
        echo $this->private;
    }
}

$obj = new MyClass();
echo $obj->public; // Fonctionne
echo $obj->protected; // Erreur fatale
echo $obj->private; // Erreur fatale
$obj->printHello(); // Affiche Public, Protected et
Private
```

7.1.4. Héritage

C'est un mécanisme qui permet à une classe fille d'hériter de toutes les méthodes, propriétés d'une classe mère.

Par exemple, pour une classe étendue, la classe enfant hérite de toutes les méthodes publiques et protégées, propriétés et constantes de la classe parente.

```
<?php

class Foo
{
    public function printItem($string)
    {
        echo 'Foo: ' . $string . PHP_EOL;
    }

    public function printPHP()
    {
        echo 'PHP est super' . PHP_EOL;
    }
}

class Bar extends Foo
{
    public function printItem($string)
    {
        echo 'Bar: ' . $string . PHP_EOL;
    }
}

$foo = new Foo();
$bar = new Bar();
$foo->printItem('baz'); // Affiche : 'Foo: baz'
$foo->printPHP();       // Affiche : 'PHP est super'
$bar->printItem('baz'); // Affiche : 'Bar: baz'
$bar->printPHP();       // Affiche : 'PHP est super'
```

7.1.5. Statique

Le fait de déclarer des propriétés ou méthodes en **static** permet d'y avoir accès sans instancier la classe. Voici un exemple de méthode **static**.

```
<?php
class Foo
{
    public static function aStaticMethod() {
        // ...
    }
}

Foo::aStaticMethod();
$classname = 'Foo';
$classname::aStaticMethod();
?>
```

Les propriétés **static** peuvent être accédées en utilisant l'opérateur de résolution de portée (::).

```
<?php
class Foo
{
    public static $my_static = 'foo';

    public function staticValue() {
        return self::$my_static;
    }
}

class Bar extends Foo
{
    public function fooStatic() {
        return parent::$my_static;
    }
}

print Foo::$my_static . "\n";

$foo = new Foo();
print $foo->staticValue() . "\n";
print $foo->my_static . "\n";          // "Propriété"
my_static non définie

print $foo::$my_static . "\n";
$classname = 'Foo';
print $classname::$my_static . "\n";

print Bar::$my_static . "\n";
$bar = new Bar();
print $bar->fooStatic() . "\n";
?>
```


7.1.6. Abstraction de classes

Il est possible en PHP de définir des méthodes et classes abstraites. Ce sont des classes qui ne peuvent pas être instanciées et toute classe qui contient une méthode abstraite doit aussi être abstraite.

Toutes les méthodes abstraites dans une classe parent doivent être définies par la classe enfant.

```
<?php
abstract class AbstractClass
{
    // Force les classes filles à définir cette méthode
    abstract protected function getValue();
    abstract protected function prefixValue($prefix);

    // méthode commune
    public function printOut() {
        print $this->getValue() . "\n";
    }
}

class ConcreteClass1 extends AbstractClass
{
    protected function getValue() {
        return "ConcreteClass1";
    }

    public function prefixValue($prefix) {
        return "{$prefix}ConcreteClass1";
    }
}
```

7.1.7. Interfaces

Les interfaces permettent de définir les méthodes qu'une classe doit implémenter, sans avoir à définir leur fonctionnement. Les interfaces sont définies en utilisant le mot-clé **interface**.

Toutes les méthodes déclarées dans une interface doivent être publiques. Voici deux principaux rôles des interfaces:

- ❖ Afin de rendre des objets de classe interchangeable car elles implémentent la ou les mêmes interfaces. Par exemple, votre application peut utiliser plusieurs services d'accès à des bases de données, plusieurs gestionnaires de paiement.
- ❖ Permettre à une fonction ou méthode d'utiliser un paramètre conforme à une interface sans se soucier de comment c'est implémenté.

Pour implémenter une interface, l'opérateur **implements** est utilisé, voici un exemple:

```
<?php

// Declaration de l'interface 'Template'
interface Template
{
    public function setVariable($name, $var);
    public function getHtml($template);
}

// Implémentation de l'interface
// Ceci va fonctionner
class WorkingTemplate implements Template
{
    private $vars = [];

    public function setVariable($name, $var)
    {
        $this->vars[$name] = $var;
    }

    public function getHtml($template)
    {
        foreach($this->vars as $name => $value) {
            $template = str_replace('{ ' . $name . ' }',
$value, $template);
        }

        return $template;
    }
}
```

Les interfaces sont aussi extensibles, c'est-à-dire qu'il est possible d'avoir une interface parent et enfant.

```
<?php
interface A
{
    public function foo();
}

interface B extends A
{
    public function baz(Baz $baz);
}

// Ceci fonctionnera
class C implements B
{
    public function foo()
    {
    }

    public function baz(Baz $baz)
    {
    }
}

// Ceci ne fonctionnera pas et entrainera une erreur
fatale
class D implements B
{
    public function foo()
    {
    }

    public function baz(Foo $foo)
    {
    }
}
```

7.1.8. Traits

C'est un mécanisme de réutilisation de code dans un langage à héritage simple tel que PHP. Un trait est comme une classe et sert à regrouper des fonctionnalités. Il est possible d'utiliser plusieurs traits dans une même classe.

```
<?php
trait Hello {
    public function sayHello() {
        echo 'Hello ';
    }
}

trait World {
    public function sayWorld() {
        echo 'World';
    }
}

class MyHelloWorld {
    use Hello, World;
    public function sayExclamationMark() {
        echo '!';
    }
}

$o = new MyHelloWorld();
$o->sayHello();
$o->sayWorld();
$o->sayExclamationMark();
?>
```

7.1.9. Parcours d'objets

PHP nous offre un foreach afin de parcourir une liste d'objets.

```
class MyClass
{
    public $var1 = 'valeur 1';
    public $var2 = 'valeur 2';
    public $var3 = 'valeur 3';

    protected $protected = 'variable protégée';
    private $private = 'variable privée';

    function iterateVisible() {
        echo "MyClass::iterateVisible:\n";
        foreach ($this as $key => $value) {
            print "$key => $value\n";
        }
    }
}

$class = new MyClass();

foreach($class as $key => $value) {
    print "$key => $value\n";
}

echo "\n";

$class->iterateVisible();
```

Recopier le code et exécuter pour voir l'affichage.

8. Formulaire avec PHP

Avec PHP il est possible d'avoir tous les champs d'un formulaire disponibles dans le script PHP d'action.

```
<form action="action.php" method="post">
  <label>Votre nom :</label>
  <input name="nom" id="nom" type="text" />

  <label>Votre âge :</label>
  <input name="age" id="age" type="number" /></p>

  <button type="submit">Valider</button>
</form>
```

Lorsque le bouton valider est cliqué le fichier `action.php` est appelé qui contient le script suivant:

```
Bonjour, <?php echo htmlspecialchars($_POST['nom']); ?>.
Tu as <?php echo (int)$_POST['age']; ?> ans.
```

Ce qui affichera

```
Bonjour Jean.
Tu as 29 ans.
```

Les variables `$_POST['nom']` et `$_POST['age']` sont automatiquement créées par PHP. Dans cet exemple nous avons utilisé la méthode `POST` mais si nous avons utilisé la méthode `GET` notre formulaire aurait placé ces informations dans la variable `$_GET`.

9. Base de données MySQL

Dans ce cours nous allons interagir avec MySQL qui est un système de gestion de bases de données relationnelles(SGBDR) écrit en C et C++.

Voici quelque syntaxe SQL que nous utiliserons souvent:

Dans cet exemple nous allons créer la table shop et insérer des données dans cette table

```
CREATE TABLE shop (
  article INT UNSIGNED DEFAULT '0000' NOT NULL,
  dealer CHAR(20) DEFAULT '' NOT NULL,
  price DECIMAL(16,2) DEFAULT '0.00' NOT NULL,
  PRIMARY KEY(article, dealer));
INSERT INTO shop VALUES
  (1, 'A', 3.45), (1, 'B', 3.99), (2, 'A', 10.99), (3, 'B', 1.45),
  (3, 'C', 1.69), (3, 'D', 1.25), (4, 'D', 19.95);
```

```
SELECT * FROM shop ORDER BY article;
```

```
+-----+-----+-----+
| article | dealer | price |
+-----+-----+-----+
|      1 | A      |  3.45 |
|      1 | B      |  3.99 |
|      2 | A      | 10.99 |
|      3 | B      |  1.45 |
|      3 | C      |  1.69 |
|      3 | D      |  1.25 |
|      4 | D      | 19.95 |
+-----+-----+-----+
```


Dans l'exemple suivant nous allons mettre à jour une ligne dans une table précise à l'aide d'une syntaxe SQL "UPDATE".

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK

```
UPDATE Customers  
SET ContactName = 'Alfred Schmidt', City = 'Frankfurt'  
WHERE CustomerID = 1;
```

par contre, pour supprimer une ligne on utilise "DELETE"

```
DELETE FROM Customers WHERE CustomerName='Alfreds Futterkiste';
```

10. Interaction avec une base de données MySQL avec PHP

10.1. Connecter PHP sur MySQL

Voici la syntaxe PHP qui permet de se connecter à une base de données MySQL

```
<?php
$pdo = new PDO("mysql:host=localhost;dbname=world",
    'my_user', 'my_password');
$pdo->setAttribute(PDO::MYSQL_ATTR_USE_BUFFERED_QUERY,
    false);

$unbufferedResult = $pdo->query("SELECT Name FROM
City");
foreach ($unbufferedResult as $row) {
    echo $row['Name'] . PHP_EOL;
?>
```

Il est toujours recommandé d'avoir un try.... catch pour prendre en charge les erreurs de connexion à la base de données

```
<?php
try {
    $dbh = new PDO('mysql:host=localhost;dbname=test',
    $user, $pass);
} catch (PDOException $e) {
    // attempt to retry the connection after some timeout
    for example
}
```

Afin que l'on puisse passer des paramètres dans nos requêtes SQL il y a ceux qu'on appelle les "prepared statements"

```
<?php
$stmt = $dbh->prepare("INSERT INTO REGISTRY (name, value)
VALUES (:name, :value)");
$stmt->bindParam(':name', $name);
$stmt->bindParam(':value', $value);

// insert one row
$name = 'one';
$value = 1;
$stmt->execute();

// insert another row with different values
$name = 'two';
$value = 2;
$stmt->execute();
?>
```

Voici une autre manière de passer les paramètres en utilisant "?"

```
<?php
$stmt = $dbh->prepare("INSERT INTO REGISTRY (name, value)
VALUES (?, ?)");
$stmt->bindParam(1, $name);
$stmt->bindParam(2, $value);

// insert one row
$name = 'one';
$value = 1;
$stmt->execute();

// insert another row with different values
$name = 'two';
$value = 2;
$stmt->execute();
?>
```

Récupérer des données avec les requêtes préparées "prepared statement"

```
<?php
$stmt = $dbh->prepare("SELECT * FROM REGISTRY where name
= ?");
$stmt->execute([$_GET['name']]);
foreach ($stmt as $row) {
    print_r($row);
}
?>
```